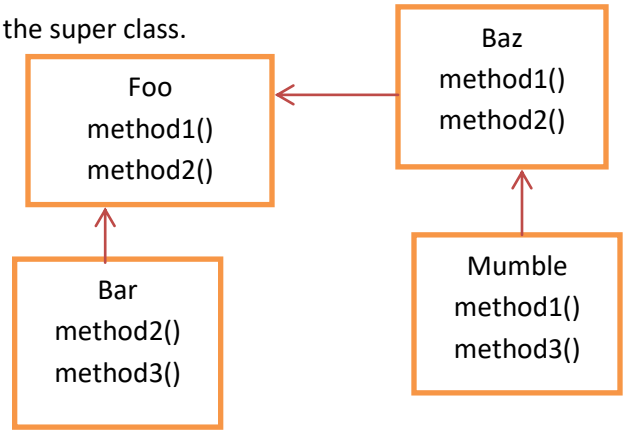
To solve this type of problems, I want you to use the following technique. For each class create a box and write the method names in it, then use an arrow to point to its super class. For this particular problem

```
class Foo {
  public void method1() {
    System.out.println("foo method 1");
  }

  public void method2() {
    System.out.println("foo method 2");
  }
}

class Bar extends Foo {
  public void method2() {
    System.out.println("bar method 2");
  }

  public void method3() {
    System.out.println("bar method 3");
  }
}

class Baz extends Foo {
  public void method1() {
    System.out.println("baz method 1");
  }

  public void method2() {
    System.out.println("baz method 2");
    method1();
  }
}

class Mumble extends Baz {
  public void method1() {
    super.method1();
    System.out.println("mumble method 1");
  }

  public void method3() {
    System.out.println("mumble method 3");
  }
}
```

We have four classes, (foo, bar, baz and mumble) each having few methods. Now let's create a box for each class and use arrows to indicate the super class.



Now let us look at the instances created.

Foo foo1 = new Bar();

Foo is the super class of Bar, so this statement is valid. We are creating an instance of Bar, and saving it inside a variable of Foo type.

Foo foo2 = new Baz();

Foo is the super class of Baz, so this statement is valid. We are creating an instance of Baz, and saving it inside a variable of Foo type.

Bar bar1 = new Bar();

this statement is definitely valid. If you don't know the explanation of its validity, then you are in serious trouble.

Baz baz1 = new Baz();

this statement is definitely valid. If you don't know the explanation of its validity, then you are in serious trouble.

Baz baz2 = new Mumble();

Baz is the super class of Mumble, so this statement is valid. We are creating an instance of Mumble, and saving it inside a variable of Baz type.

Object obj1 = new Bar();

Object is the parent class of all classes and this is defined in java. Therefore this statement is valid.

Foo foo3 =new Mumble();

valid.

Mumble baz3 = new Baz();

Invalid. Compile time error: incompatible types: SuperClass cannot be converted to SubClass
Error: incompatible types: Baz cannot be converted to Mumble

// Method calling from Foo class

Now let's run the code line by line. Not all the following lines are correct. Therefore while you are testing it in Dr.Java or in any other IDE, comment all the lines except the line you are working with. For example, when you are trying to find out the output of the first line, do the following:

```
foo1.method1();
//foo2.method1();
//bar1.method1();
//baz1.method1(); ..
```

Let us run the first line and see whether it produces a valid output, compile error or a run time error.
foo1.method1();
From the objects created above, **foo1** is the variable name for the class type **foo** containing the object of **bar**. It's calling **method1**(). Now refer to the box of **bar** created above. Check if it contains the method **method1**(). If it does, do as it says and if **method1**() does not exist, check if its parent class has the method or not. To go to **bar's** parent class check the arrow that is coming out of the box. It points to **foo** and inside the **foo** box there is **method1**() and therefore **foo's method**()1 will be invoked and the output will be **`foo method 1.`**

foo2.method1();

Let us comment the first line and uncomment the second line and then run the second line and see whether it produces a valid output, compile error or a run time error.
foo2.method1();
foo2 is the variable name for the class type **foo** containing the object of **baz**. It's calling **method1**(). Now refer to the box of **baz** created above. Check if it contains the method **method1**(). If it does, do as it says and if **method1**() does not exist, check if its parent class has the method or not. In this case **baz** has **method1**() and therefore its own **method**()1 will be invoked and the output will be **baz method 1.**

bar1.method1();

bar1 is the variable name for the class type **bar** and it's calling method1().Now refer to the box of **bar** created above. Check if it contains the method **method1**(). If it does, do as it says and if **method1**() does not exist, check if its parent class has the method or not. To go to **bar's** parent class check the arrow that is coming out of the box. It points to **foo** and inside the **foo** box there is **method1**() and therefore **foo's method**()1 will be invoked and the output will be **`foo method 1`**.

baz1.method1();

baz1 is the variable name for the class type **baz** and it's calling method1().Now refer to the box of **baz** created above. Check if it contains the method **method1**().In this case **baz** has **method1**() and therefore its own **method**()1 wil-l be invoked and the output will be **baz method 1.**

baz2.method1();

baz2 is the variable name for the class type **baz** containing the object of **mumble**. It's calling **method1**().Now refer to the box of **mumble** created above. Check if it contains the method **method1**(). If it does, do as it says and if **method1**() does not exist, check if its parent class has the method or not. **Mumble** has **method1**() and Inside it, there is a line **super.method1**();. This means the **method1**() of its super class will be invoked. Go to its super class and check for **method1**(). If it exists, run it and if the super class does not have such method, a compile time error will occur.

In this case the super class of **mumble**, **baz**, has **mehod1**() and therefore and the output will be **baz method 1**. But wait!! This is not it. Have you noticed, in **mumble**, after the line **super.method1(),** there is another line that says **System.out.println("mumble method 1");** Therefore there will be another output **mumble method 1.**

obj1.method1();

obj1 is the variable name for the class type **Object** containing the object of **bar**. It's calling **method1**().Now refer to the box of **bar** created above. Check if it contains the method **method1**(). If it does, do as it says and if **method1**() does not exist, check if its parent class has the method or not. In this case, the object of **bar** is stored inside **Object**. As **bar** does not have **method1**(), **it will look inside Object to find method1(). Object does not have anything called method1() and therefore it will be a compile time error.**

foo1.method2();

foo1 is the variable name for the class type **foo** containing the object of **bar**. It's calling **method2**(). Now refer to the box of **bar** created above. Check if it contains the method **method2**(). If it does, do as it says and if **method2**() does not exist, check if its parent class has the method or not. There is method2() in bar and therefore the output will be **bar method 2.**

*****foo2.method2();**

foo2 is the variable name for the class type **foo** containing the object of **baz**. It's calling **method2**(). Now refer to the box of **baz** created above. Check if it contains the method **method2**(). If it does, do as it says and if **method2**() does not exist, check if its parent class has the method or not. In this case **baz** has **method2**() and therefore its own **method2**() will be invoked and the output will be **baz method 2.** This is not it. There is another line saying, **method1**(). Therefore its **method1**() will be called and there will be another output **baz method 1;**

bar1.method2();

bar1 is the variable name for the class type **bar** and it's calling method2().Now refer to the box of **bar** created above. Check if it contains the method **method2**(). If it does, do as it says and if **method2**() does not exist, check if its parent class has the method or not**. Method2()** exists and the output will be **bar method 2.**

baz1.method2();

baz1 is the variable name for the class type **baz** and it's calling **method2**().Now refer to the box of **baz** created above. Check if it contains the method **method2**() **.** It does contain and therefore the output will be **baz method 2.** Then again, there is another line **method1()** and **method1**() of **baz** will be called. The second output will be **baz method 1**.

baz2.method2();

baz2 is the variable name for the class type **baz** containing the object of **mumble**. It's calling **method2**().Now refer to the box of **mumble** created above. Check if it contains the method **method2**(). If it does, do as it says and if **method2**() does not exist, check if its parent class has the method or not. **Mumble** does not have anything called **mehod2**() and therefore it goes to his super class **baz**. **Baz** has **method2**() and therefore and it will be called and it will first output **baz method 2.** now notice there is another line **method1**().

Now the question is, both **baz** and **mumble** has **method1**(), whose method will be called? The answer is, **always work with a class's own method. Go to the super class only if you don't find it in the sub class.** Therefore ***method1()*** *of mumble class* will be called. Now once you go to the **method1**() of mumble, there is another line *saying* ***super.method1()***. Now you have to go to its super class **baz** and call its **method1**() and the second output will be **baz method 1**. This is not the end, in mumble after the line **super**.**method1**() there is another line where it asks you to print something, Hence there is a third output and it is **mumble method 1**.

foo3.method2();

no compiler error coz ref Foo class has method2(). in Run time: calling parent Baz class not referent Foo.

Foo Moo output
baz method 2
baz method 1
mumble method 1

If Baz does not have method2, then Foo's method2 will be called.

foo3.method4();

compiler error coz neither Mumble nor ref Foo class has method4(). parent class is searched for method4() until run time**.**

✓ *Casting, Compile and Run time error:*

((Baz)obj1).method2();

Let us see ((Baz)obj1) part first. Obj1 is an Object type and as we know all classes extend Object, casting is valid in here. This part of the code will show **no error in compilation.**

Now let us see the .**method2**() part. As **obj1** as has been casted to **Baz** , and **Baz** has **method2**() there is nothing wrong with this line. **No compile time error.**

Now run it. **ALAS!!!** It shows a **run time error**. But why? Look at this line carefully. **Object obj1 = new Bar(); obj1** has an instance of **Bar**, not **Object**., and **Bar** and **Baz** has no connection at all. Therefore casting was not done and therefore it shows a **run time error.**

((Object)bar1).method3();

As Object is the super class of all classes, the part **((Object)bar1)** is valid, and bar1 has been cast to Object. Does the super class Object have anything called method3()? No, **it will show an compile time error.**

**** ((Foo)baz2).method3();**

As Foo is the super class of baz, the part **((Foo)baz2).**is valid, and baz2 has been cast to Foo. Does the super class Foo have anything called method3()? No, **it will show an compile time error.**

((Foo)baz2).method2();

No compile time error . No run time error.
Output:
baz method 2
baz method 1
mumble method 1

((Foo)baz2).method1();

No compile time error . No run time error.
Output:
baz method 1
mumble method 1

((Bar)foo1).method3();

No compile time error . No run time error. Foo1 contains the object of **bar**. **((Bar)foo1)** therefore casts **bar** into **bar** which is valid. **Bar** contains **method3**() and therefore the output will be **bar method 3.**

((Mumble)baz1).method3();

Mumble contains method3() so this **will compile successfully**. **baz1** contains the object of **baz** itself and mumble is its subclass. **A super class cannot be casted into its subclass (exception is class Object)** so it will throw a run time **exception**.

((Mumble)baz2).method3()

Mumble contains method3() and **this will compile successfully**. Baz2 has the instance of mumble. **((Mumble)baz2)** casts a mumble object into a mumble. Therefore it **will run smoothly**. Output: mumble method 3

((Baz)foo2).method3()

Baz does not have method3() nor does its superclass foo. Therefore **it will never compile.**

((Bar)foo2).method2();

bar has method2() **so it will compile**. Foo2 has the object of baz , baz and bar has no relation. These two classes are totally separate and thus cant be cast. Therefore there will be **a run time error.**

((Foo)obj1).method2();

foo has method2() and therefore **it will compile**. Obj1 has bar and as foo is the super class of bar the casting is successful. This piece of **code will run smooth**. Output: bar method 2

Note:

- At compile time Casting is permitted if both are in the same branch of the tree you have drawn. If the casting is permitted, the compiler then checks if the class has the specific method. If no, then does any of the classes it's inheriting from have the method? If no, then a compiler error occurs. If yes, then the compiler hands it over to the runtime.

- At runtime, the actual object creation with the 'new' keyword happens. During runtime if casting is successful then, the method on the actual object is called. If it does not have the method, the request propagates up the hierarchy until one of the parents have the method and responds with the output.